

Software Engineering II
Semester 2026-I
Workshop No. 3 — Full Stack Implementation and Testing

Eng. Liliana Marcela Olarte, M.Sc.
Eng. Carlos Andrés Sierra, M.Sc.

Lecturers
Computer Engineering Program
School of Engineering
Universidad Nacional de Colombia

Welcome to Workshop 3! This session focuses on the *implementation* of your project's database, backend services (Java with **Spring Boot** and Python with **SQLAlchemy**), mobile frontend using **react-native**, REST API documentation with **Swagger**, and unit testing. The goal is to achieve a working, integrated system with proper ORM usage and ensure code quality through testing.

Scope and Objectives

- **Database Implementation:** Design and implement a relational database schema using PostgreSQL or MySQL with proper relationships, keys, and normalization.
- **Backend Implementation:** Develop backend services in Java using **Spring Boot** with JPA/Hibernate ORM, and in Python using **FastAPI** or **Flask** with **SQLAlchemy** ORM.
- **REST API Integration:** Expose RESTful endpoints from both backends with comprehensive **Swagger/OpenAPI** documentation and interactive Swagger UI.
- **Mobile Frontend:** Implement a mobile application using **react-native** that interacts with your backend APIs.
- **Unit Testing:** Write and execute unit tests for backend services using **JUnit** (Java) and **pytest** (Python).

Carlos Andrés Sierra, Computer Engineer, M.Sc. in Computer Engineering, Lecturer at Universidad Nacional de Colombia.

Any comment or concern regarding this workshop can be sent to Carlos A. Sierra at: *casier-rav@unal.edu.co*.

Methodology and Deliverables

1. Database Implementation

- Provide SQL scripts or migration files for creating all tables, keys, and relationships.
- Include sample data for testing.
- Document the schema in your `README.md`.

2. Backend Services

- Deliver source code for both `Java` and `Python` backends.
- Organize code in folders named `java-backend` and `python-backend`.
- Include configuration files for database connections.
- Document all REST API endpoints with `Swagger/OpenAPI` specifications with example requests and responses.
- Provide interactive API documentation accessible via Swagger UI.

3. Mobile Frontend

- Implement a mobile application using `react-native` that consumes your backend APIs.
- Provide screenshots or demo videos showing main features.
- Organize code in a folder named `react-native-frontend`.

4. Unit Testing

- Write unit tests for backend functions using `JUnit` (`Java`) and `pytest` (`Python`).
- Include test results or screenshots showing successful execution.

5. Integration Evidence

- Show how the frontend interacts with both backends via REST APIs.
- Provide evidence such as code snippets, screenshots, or demo videos.

6. Delivery Format

- Organize all files in a folder named `Workshop-3` in your course project repository.
- Provide a `README.md` referencing each section and explaining setup and usage.

Project Requirements Checklist

- Working database schema with proper relationships and sample data.
- `Java` backend with `Spring Boot` and `JPA/Hibernate` (e.g., authentication, user management).
- `Python` backend with `SQLAlchemy` (e.g., business logic, CRUD operations).

- REST API endpoints documented using **Swagger/OpenAPI** and accessible via Swagger UI.
- Mobile frontend using **react-native** interacting with backend APIs.
- Unit tests for both backends (JUnit for Java, pytest for Python).
- Organized and referenced documentation including API specifications.

Recommended Technology Stack

This workshop recommends the following technology stack for optimal learning and industry alignment:

- **Database:** Relational database using PostgreSQL or MySQL with proper schema design, relationships, and indexing.
- **Java Backend:** Spring Boot framework with JPA/Hibernate as the Object-Relational Mapping (ORM) layer, JUnit for unit testing, and Swagger/Springdoc for API documentation.
- **Python Backend:** FastAPI or Flask framework with SQLAlchemy as the Object-Relational Mapping (ORM) layer, pytest for unit testing, and Swagger/OpenAPI for API documentation.
- **Mobile Frontend:** react-native for cross-platform mobile application development.
- **API Communication:** REST principles with JSON data format.

Deadline

Monday, July 20th, 2025, at 12:00. Late submissions will not be accepted.

Notes

- All documents must be in **English**.
- Cite any references (**articles, tutorials, tools**) that influenced your design choices.
- Focus on clarity and completeness. This implementation phase will guide your progress in future workshops.

Good luck! A robust backend, working frontend, and thorough testing will set the stage for a successful project implementation.